Project - High Level Design

Retail Rag

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Tamanna Katariya	EN22CS3011013
2.	Tanishka Patidar	EN22CS3011018
3.	Vivek Dhawle	EN22CS3011097
4.	Yogesh Ramani	EN22CS3011122

Group Name: 08 D9-D10

Project Number: GAI-01

Industry Mentor Name: Mr. Rishabh Tripathi

University Mentor Name: Prof. Divya Kumrawat

Academic Year: 2025-2026

Table of Contents

1. Introduction :

1.1. Scope of the document.

1.2. Intended Audience

1.3. System overview.

2. System Design :

2.1. Application Design

2.2. Process Flow

2.3. Information Flow

2.4. Components Design

2.5. Key Design Considerations

2.6. API Catalogue.

3. Data Design :

3.1. Data Model

3.2. Data Access Mechanism

3.3. Data Retention Policies

3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements :

7.1. Security Aspects

7.2. Performance Aspects

8. References

1. Introduction

The increasing volume of retail data such as reports, invoices, manuals, and customer documents has made it difficult to extract meaningful insights efficiently. Traditional keyword-based search systems fail to provide context-aware and accurate results when dealing with unstructured data.

To solve this, the project **"Retail Rag – Chat with Retail Documents"** uses a **Retrieval-Augmented Generation (RAG)** approach. The system enables users to upload documents and interact with them using natural language queries. By combining vector databases with Large Language Models (LLMs), the system retrieves relevant document content and generates accurate, context-aware responses.

The rapid growth of digital transformation in the retail sector has resulted in the generation of massive amounts of unstructured data. These documents often contain valuable business insights, but extracting relevant information manually is time-consuming and inefficient.

The Retail Rag system addresses this challenge by enabling intelligent interaction with documents using natural language processing techniques. Instead of manually searching through files, users can directly ask questions and receive precise answers derived from the document content. This significantly improves productivity, decision-making, and user experience.

1.1 Scope of the document

This document provides the high-level architecture and system design of the Retail Rag platform. It includes:

- Overall system architecture
- Document ingestion and processing workflow
- Integration with LLM and vector database
- Query processing pipeline
- Key design decisions and scalability considerations

1.2 Intended Audience

- Developers and Engineers
- Project Evaluators and Faculty
- Business Users and Analysts
- AI/ML Enthusiasts

1.3 System Overview

The system allows users to interact with retail documents through a conversational interface. The system is designed to bridge the gap between raw document storage and intelligent information retrieval. It leverages semantic understanding rather than keyword matching, ensuring that even indirectly related queries return meaningful results.

The integration of vector embeddings allows the system to understand context, while the Large Language Model enhances response generation by synthesizing retrieved information into human-readable answers.

Workflow Overview:

1. User uploads document (PDF, DOCX)
2. Text is extracted and chunked
3. Each chunk is converted into embeddings
4. Embeddings are stored in vector database
5. User submits query
6. System retrieves relevant chunks
7. LLM generates final response

Key Features:

- Context-aware Q&A
- Reduced hallucination using RAG

- Fast semantic search
- Scalable architecture

2. System Design

2.1 Application Design :

This layered architecture ensures modularity, scalability, and maintainability. Each layer operates independently, allowing easy updates and integration of new technologies without affecting the overall system.

1. Presentation Layer (Frontend)

- This layer focuses on user interaction and usability.
- Provides UI for:
  - Document upload
  - Chat interface
- Technologies: React / HTML / CSS

---

2. Application Layer (Backend)

- This layer acts as the core processing unit where all logic is executed.
- Handles:
  - Business logic
  - Document processing
  - Query orchestration
- Technologies: Python (FastAPI)

---

3. Data Layer

- This layer ensures efficient storage and retrieval of structured and unstructured data.

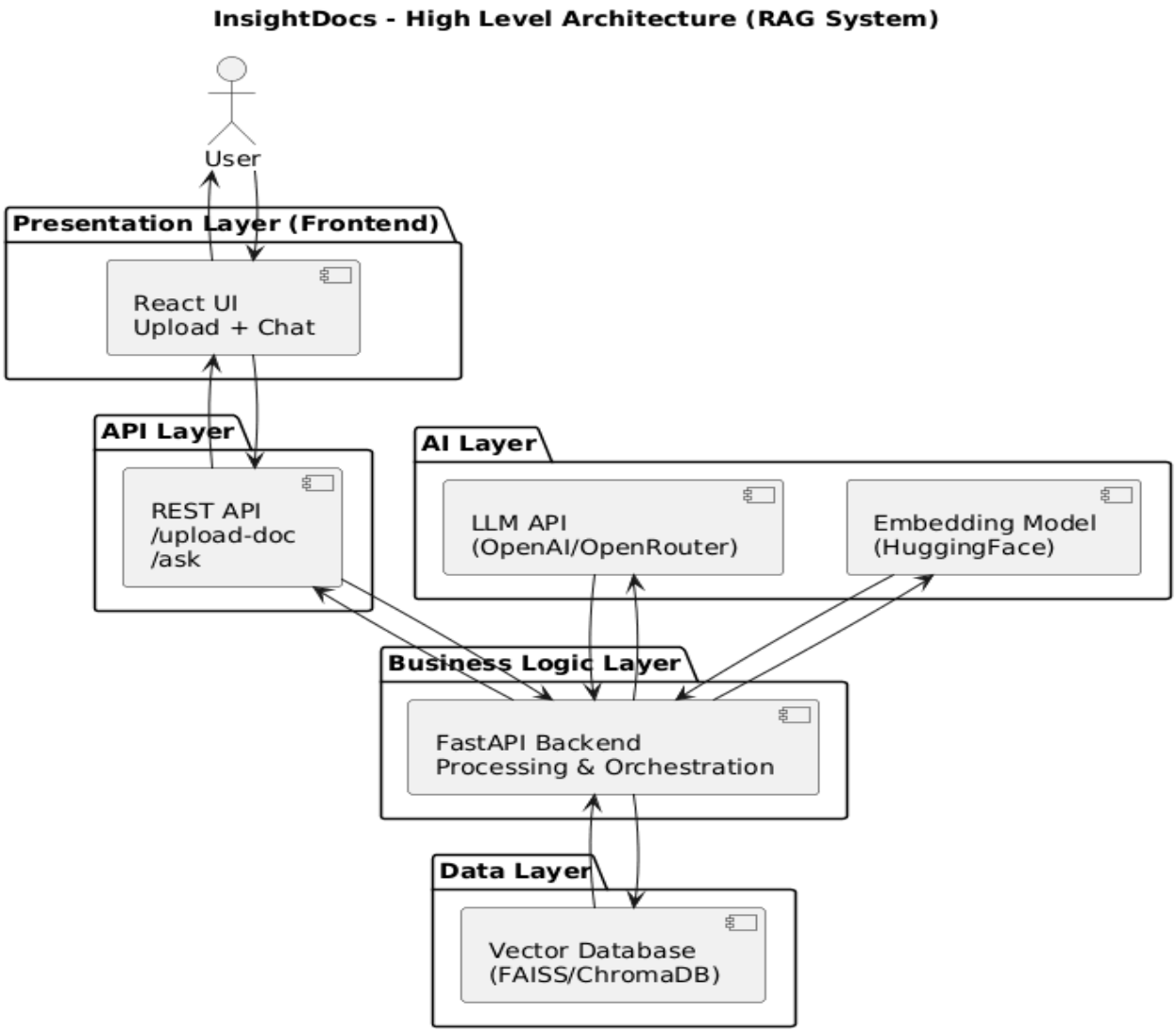
- Stores:
  - Document metadata
  - Embeddings
- Technologies:
  - Vector DB (FAISS / ChromaDB)

---

4. AI Layer

- This layer provides intelligence through embeddings and language understanding.

- Embedding model
- LLM API (for response generation)



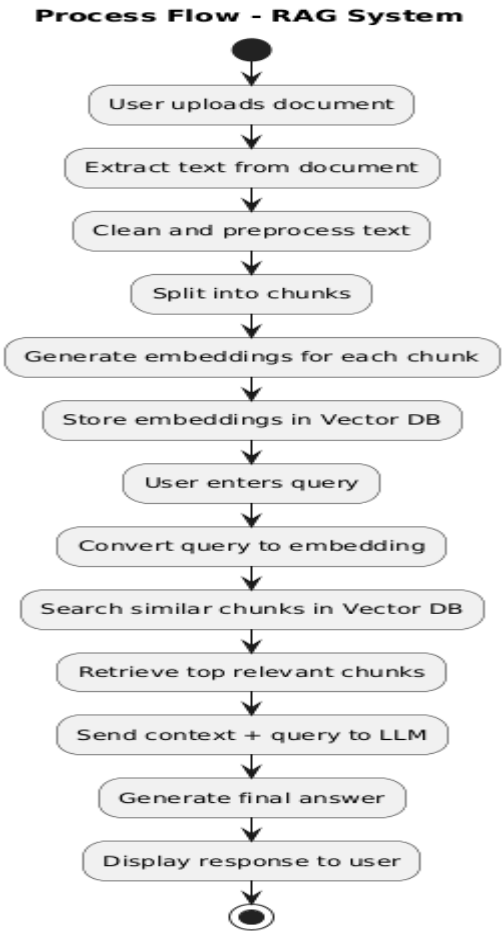
2.2 Process Flow:

The process flow is divided into two main phases: document ingestion and query handling. This separation ensures that heavy processing tasks like embedding generation are done only once during upload, improving overall system efficiency during query time.

Additionally, the system ensures that only the most relevant document chunks are passed to the LLM, reducing computation cost and improving response accuracy.

1. Document Processing Phase

- Upload document
- Extract text
- Split into chunks
- Generate embeddings
- Store in vector DB



2. Query Processing Phase

- User inputs query
- Convert query → embedding
- Perform similarity search
- Retrieve relevant chunks
- Pass context to LLM
- Generate answer

Input:

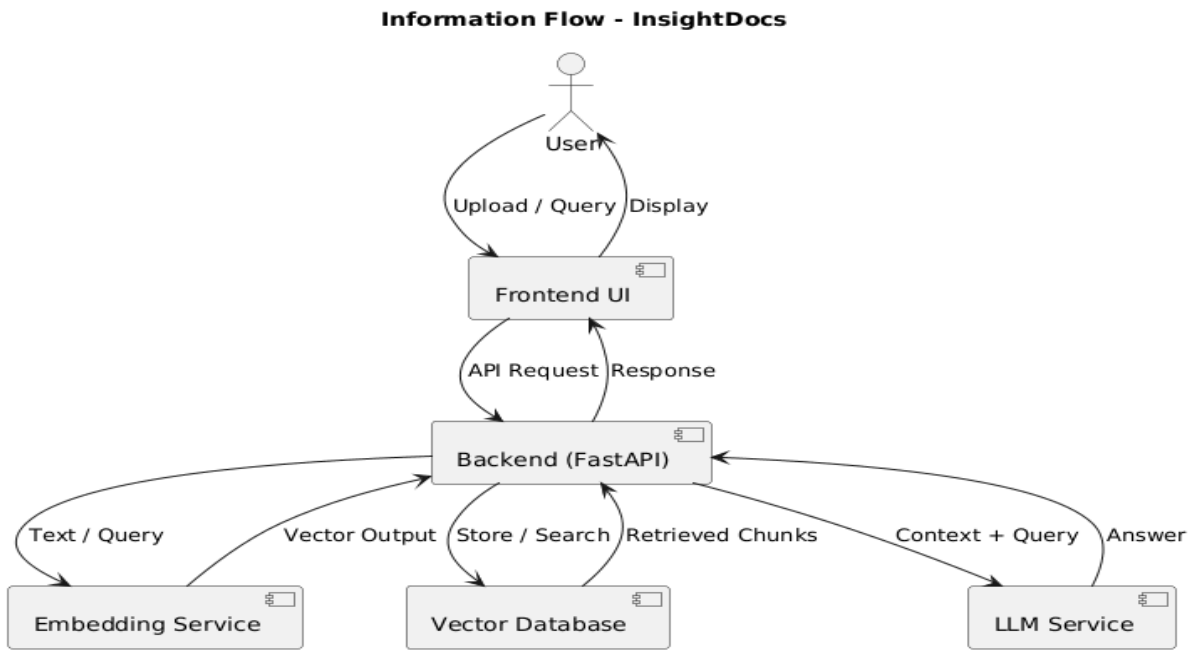
- Documents (PDF, DOCX)
- User queries

Processing:

- Text extraction
- Chunking
- Embedding generation
- Similarity search

Output:

- Context-aware response

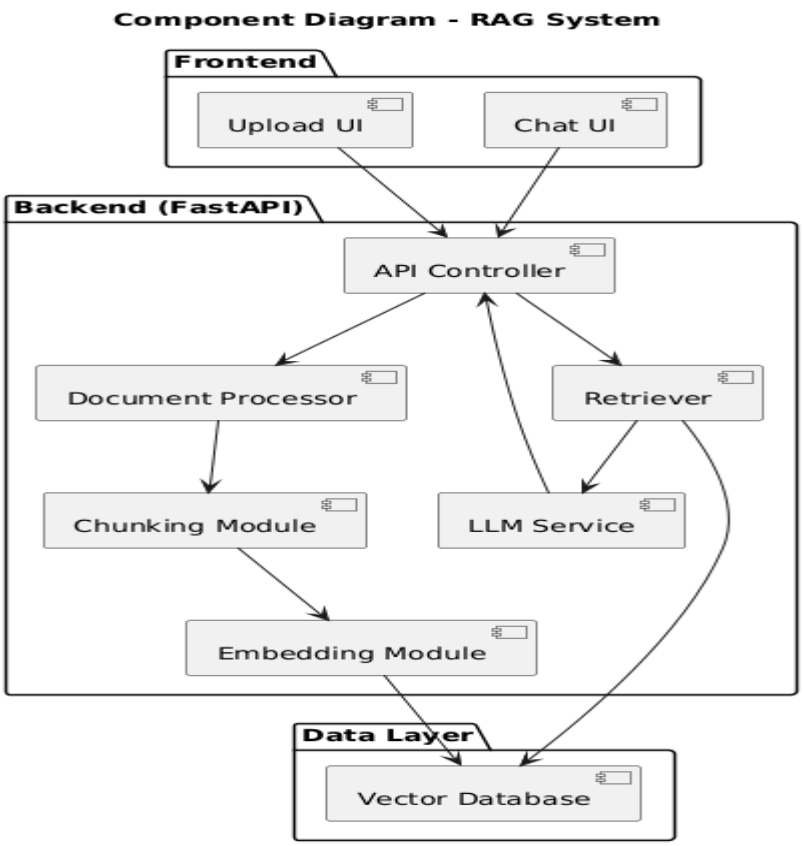


2.3 Components Design:

Each component in the system is designed to perform a specific function, ensuring separation of concerns:

Core Components:

1. **Document Loader** supports multiple file formats and ensures proper parsing.
2. **Text Splitter** optimizes chunk size to balance context and performance.
3. **Embedding Module** transforms text into high-dimensional vectors for semantic comparison.
4. **Vector Database**
   - Stores embeddings
   - Performs similarity search
5. **Retriever** plays a critical role in filtering only relevant data.
6. **LLM Service** enhances user interaction by generating natural and context-aware responses.
7. **User Interface**
   - Chat-based interaction



2.4 Key Design Considerations :

1. Scalability

- The system is designed to handle increasing data volume and concurrent users by using efficient indexing and scalable vector databases.

2. Accuracy

- By combining retrieval with generation, the system minimizes incorrect or fabricated responses, ensuring higher reliability.

3. Performance

- Preprocessing documents and storing embeddings reduces runtime computation, enabling faster query responses.

4. Security

- Proper authentication and secure storage mechanisms are implemented to protect sensitive documents.

5. Usability

- The chat-based interface simplifies interaction, making the system accessible even to non-technical users.

2.5 API Catalogue :

1. Upload Document

- Endpoint: `/upload-doc`
- Method: POST

---

2. Query API

- Endpoint: `/ask`
- Method: POST

Request:

```
{
 "query": "What is revenue growth?"
}
```

Response:

```
{
 "answer": "Revenue growth is..."
}
```

3. Login

- Endpoint: `/login`
- Method: POST

---

4. Signup

- Endpoint: `/signup`
- Method: POST

---

5. Health Check

- Endpoint: `/health`
- Method: GET

---

3. Data Design

3.1 Data Model :

The data model is structured to maintain clear relationships between documents, text chunks, and embeddings. This ensures efficient retrieval and traceability of information.

Each document is broken down into smaller chunks, which are individually indexed and linked back to the original document. This enables precise and context-aware responses.

1. Document Metadata

- Document_ID
- File_Name
- Upload_Date
- File_Type

---

2. Text Chunks

- Chunk_ID
- Document_ID
- Text_Content
- Chunk_Index

---

3. Embeddings

- Chunk_ID
- Embedding_Vector
- Vector_Dimension

---

4. Query Logs

- Query_ID
- User_Query

- Response
- Timestamp

3.2 Data Access Mechanism:

The system uses cosine similarity to measure the closeness between query embeddings and stored document embeddings. This allows semantic matching instead of exact keyword matching, improving search quality.

Efficient indexing techniques are used to ensure fast retrieval even when dealing with large datasets.

It also provides:

- API-based access
- Vector similarity search (cosine similarity)
- Indexed retrieval (FAISS / ChromaDB)

3.3 Data Retention Policies:

Data retention policies are implemented to manage storage efficiently and ensure compliance with data handling standards.

Temporary storage of embeddings reduces unnecessary resource usage, while query logs help in improving system performance and user experience through analytics.

It also provides:

- Documents stored for limited duration
- Embeddings deleted with document
- Query logs stored for analytics

3.4 Data Migration

- Supports migration from local → cloud
- Vector DB switch (FAISS → Pinecone)
- Backup & validation mechanisms

4. Interfaces:

The system interfaces are designed to ensure smooth communication between different components. REST APIs act as a bridge between the frontend and backend, while internal integrations handle communication with vector databases and AI services.

This loose coupling between components enhances flexibility and maintainability.

The functionalities are:

- Frontend ↔ Backend via REST APIs
- Backend ↔ Vector DB
- Backend ↔ LLM APIs

---

5. State and Session Management:

Although the APIs are stateless, session management is implemented to maintain conversation context. This allows the system to provide more relevant and coherent responses in multi-turn conversations.

Session IDs help track user interactions and improve personalization.

The system provides the following state and session management functionalities:

- Stateless APIs
- Session-based tracking for user queries
- Unique session IDs

---

6. Caching :

Caching is implemented in the system to improve performance, reduce latency, and optimize resource utilization. Since the system processes repeated queries and frequently accessed document data, caching helps avoid redundant computations and speeds up response generation.

The system can cache frequently asked queries along with their generated responses. When a user submits a similar query, the system can directly return the cached response instead of reprocessing the entire pipeline, thereby reducing response time.

A caching layer such as Redis can be integrated to provide fast in-memory data storage. This ensures quick read and write operations, significantly improving system responsiveness in high-load conditions.

The caching mechanism is designed to be dynamic, where cached data is periodically refreshed or invalidated to ensure that users always receive accurate and up-to-date information.

Key functionalities include:

- Caching of frequent queries and responses
- Temporary storage of embeddings and retrieved data
- Reduced latency and faster response time
- Optional integration with Redis for high-speed caching

---

7. Non-Functional Requirements :

Non-functional requirements define the quality attributes of the system that ensure reliability, efficiency, and security. These requirements focus on how the system performs rather than what it does.

The Retail Rag system is designed to meet industry standards for performance, security, and scalability, ensuring a smooth and reliable user experience.

7.1 Security :

Security is a critical component of the system, especially since it handles potentially sensitive retail documents. All communication between the client and server is secured using HTTPS to prevent data interception. Authentication mechanisms such as JWT (JSON Web Tokens) are used to verify user identity and restrict unauthorized access.

Uploaded documents are handled securely, ensuring that files are stored safely and are only accessible to authorized users. Proper validation is performed during file upload to prevent malicious inputs.

Key security features include:

- Secure communication using HTTPS
- JWT-based authentication and authorization
- Secure document upload and storage
- Input validation to prevent malicious data
- Protection of sensitive information

7.2 Performance

Performance is a key requirement to ensure that users receive fast and accurate responses. The system is optimized to handle large volumes of data and multiple user queries efficiently.

The use of vector databases enables fast similarity search, allowing the system to quickly retrieve relevant document chunks. This reduces the time required for information retrieval and enhances overall system performance.

Caching mechanisms further improve performance by storing frequently accessed data and reducing redundant processing.

The system is designed to maintain low latency even under increasing load, and it can be scaled horizontally to support more users and larger datasets.

Key performance features include:

- Fast response using vector similarity search
- Low latency query processing
- Efficient document preprocessing
- Support for multiple concurrent users
- Scalability for growing data and usage

---

8. References

- RAG Paper (NeurIPS 2020)
- OpenAI API Docs
- FAISS Documentation
- LangChain Docs